

## Problem Statement #42 – Internal Microservice Catalog & Health Portal

**Scenario:** An enterprise developer portal that maps microservice dependencies, aggregates API documentation, and performs automated periodic health checks with downtime alerts.

| Req ID | Type       | Description                                                                                                                                                    | Priority | Acceptance Criteria                                                                                                                                                              | Rationale                                                                                            |
|--------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| FR-001 | Functional | The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.                       | High     | Pass: Health probes are generated every 30 seconds and the system displays a rolling 24-hour availability percentage. Fail: No periodic probe or no availability calculation.    | Ensures continuous monitoring and provides a measurable view of service reliability.                 |
| FR-002 | Functional | The system shall allow a DevOps Engineer to register and update microservice details, including the service name, health endpoint, and dependency information. | High     | Pass: A valid service can be registered and its details can be updated and retrieved from the catalog. Fail: Required service information cannot be saved or updated.            | Provides the service catalog with the information required for monitoring and dependency management. |
| FR-003 | Functional | The system shall display an interactive dependency graph showing relationships among registered microservices.                                                 | High     | Pass: The dependency graph displays registered services and their configured relationships and can be interacted with. Fail: A configured dependency is missing from the graph.  | Helps System Architects understand service architecture and identify dependencies.                   |
| FR-004 | Functional | The system shall aggregate and display API documentation for registered microservices.                                                                         | Medium   | Pass: Selecting a registered service displays its available API documentation through the portal. Fail: The documentation is unavailable for a configured service.               | Provides developers and architects with a centralized view of service APIs.                          |
| FR-005 | Functional | The system shall record downtime and dispatch an alert when a microservice has three consecutive failed health probes.                                         | High     | Pass: After three consecutive failed probes, downtime is recorded and an alert is dispatched. Fail: An unreachable service is marked healthy or no alert is sent after the third | Enables timely detection and notification of service outages.                                        |

|         |                                        |                                                                                                                                                            |      |                                                                                                                                                                                                                                                                                 |                                                                                                 |
|---------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|         |                                        |                                                                                                                                                            |      | failure.                                                                                                                                                                                                                                                                        |                                                                                                 |
| NFR-001 | Nonfunctional – Performance & Security | The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.                             | High | Pass: Benchmarking confirms that a dependency graph containing up to 200 services renders interactively within the defined performance target under simulated peak load and required security controls. Fail: The graph becomes unusable or fails the required security checks. | Supports practical architectural visualization at the scale specified in the problem statement. |
| NFR-002 | Nonfunctional – Reliability            | The system shall maintain accurate health-check and downtime records for all monitored microservices without losing probe results during normal operation. | High | Pass: In a controlled test of 1,000 scheduled health probes, 100% of probe outcomes and timestamps are retained and downtime events are correctly recorded. Fail: Any probe result or required downtime record is lost.                                                         | Ensures availability calculations and downtime alerts are based on complete monitoring data.    |

|                                |                                                                                                                                                 |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>             | UC-03                                                                                                                                           |
| <b>Use Case Name</b>           | Monitor Microservice Health                                                                                                                     |
| <b>Primary Actor</b>           | DevOps Engineer                                                                                                                                 |
| <b>Supporting Actor/System</b> | Microservice Health Monitoring Service                                                                                                          |
| <b>Purpose</b>                 | Monitor registered microservices through periodic health probes, record results, calculate rolling 24-hour availability, and identify downtime. |

## **1. Preconditions**

The DevOps Engineer is authenticated in the portal.

The microservice is registered in the service catalog.

A valid health endpoint is configured for the microservice.

The health-monitoring service is active.

## **2. Postconditions**

The latest health-check result is recorded with its timestamp.

The microservice's rolling 24-hour availability percentage is updated.

If downtime is detected, the downtime is recorded and an alert is dispatched.

## **3. Main Success Scenario**

The DevOps Engineer opens the Internal Microservice Catalog & Health Portal.

The system displays registered microservices and their current health status.

The system sends a health probe to the configured microservice health endpoint.

The microservice returns a successful health response.

The system records the successful health-check result and timestamp.

The system updates the microservice's availability statistics.

The system calculates the rolling 24-hour service availability percentage.

The system displays the updated health status and availability percentage.

The system waits until the next scheduled health-check interval.

The system repeats the health check at 30-second intervals.

## **4. Alternate Flow – Microservice Downtime**

At Step 4, the health endpoint does not return a successful response.

The system records the failed health probe with its timestamp.

The system performs the next scheduled health probe.

If three consecutive health probes fail, the system marks the microservice as Down.

The system records the downtime period.

The system dispatches a downtime alert to the relevant stakeholder.

The system updates the rolling 24-hour availability percentage.

The system displays the microservice as unavailable in the portal.

## 5. Requirement Alignment

The flow uses the scenario's supplied monitoring behavior: health endpoints are pinged at 30-second intervals and rolling 24-hour service availability percentages are calculated. The supplied acceptance criterion requires downtime to be recorded and an alert dispatched after three consecutive failed health probes; an unreachable service must not be marked healthy.